

**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

**Metody i środki próbkowania zbiorów danych i ich
preprocesingu a wpływ na wydajność algorytmów
wnioskowania**

Nina Magdoń 176832, Kacper Lis 176828
grupa laboratoryjna : L - 4

Sprawozdanie z projektu

Opiekun pracy:
Dr inż. Marek Bolanowski

Rzeszów, 2025

Spis treści

1. Wstęp.....	4
2. Techniki w zakresie próbkowania danych, preprocesingu oraz zastosowania algorytmu drzewa decyzyjnego	5
2.1. Metody i rozwiązania.....	5
a) Preprocesing i próbkowanie	5
b) Drzewo decyzyjne	6
2.2. Dyskusja i krytyczna ocena stanu aktualnego.....	6
2.3. Podsumowanie stanu wiedzy	6
3. Eksperyment: wpływ metod próbkowania i preprocesingu na skuteczność drzewa decyzyjnego	7
3.1. Założenia i dane	7
3.2. Metodyka eksperymentu.....	7
3.2.1 Etap 1 – Przygotowanie danych.....	7
3.2.2 Etap 2 – Próbkowanie	7
3.2.3 Etap 3 – Uczenie i testowanie	8
3.3. Wyniki i ich interpretacja	8
3.3.1 Wpływ próbkowania na jakość klasyfikacji	8
3.3.2 Struktura drzewa i interpretacja	11
3.3.3 Czas wykonania i złożoność obliczeniowa :.....	12
3.4. Podsumowanie części eksperymentalnej	12
4. Podsumowanie i wnioski końcowe	13
Zalecenia i możliwe modyfikacje:	13
Wkład własny autora:	13
Autor za własny wkład pracy uważa:	13
Załączniki.....	14
Załącznik A – Pliki źródłowe programu (Python):.....	14
Literatura.....	15

1. Wstęp

W dobie gwałtownego wzrostu ilości generowanych danych rośnie również znaczenie metod umożliwiających efektywne i wiarygodne przetwarzanie informacji. Szczególnie istotne staje się to w kontekście analizy dużych zbiorów danych (ang. *big data*), w których nie tylko sam proces przetwarzania, ale także przygotowanie danych wejściowych ma bezpośredni wpływ na skuteczność i wydajność systemów wnioskowania. Problem ten znajduje swoje zastosowanie m.in. w systemach detekcji anomalii, cyberbezpieczeństwa czy predykcji zachowań użytkowników, gdzie dokładność oraz czas reakcji algorytmów decydują o ich praktycznej wartości.

Coraz częściej w badaniach naukowych i publikacjach pojawiają się analizy dotyczące preprocesingu danych – obejmującym m.in. oczyszczanie danych, transformację cech, a także metody próbkowania. Właściwy dobór metod przetwarzania wstępnego oraz odpowiednie zredukowanie lub zbalansowanie zbioru danych może znacząco wpłynąć na wyniki końcowe modelu, nie tylko w zakresie jego dokładności, ale również efektywności obliczeniowej. Szczególną uwagę badaczy przyciąga próbkowanie danych w przypadkach, gdy mamy do czynienia z niezbalansowanymi zbiorami lub dużą liczbą przykładów negatywnych, jak ma to miejsce w danych pochodzących z obszaru cyberbezpieczeństwa.

W niniejszej pracy skoncentrowano się na analizie wpływu wybranych metod próbkowania i preprocesingu danych na skuteczność działania algorytmu drzewa decyzyjnego (*decision tree*). Analizie poddano dane pochodzące z rzeczywistego i szeroko wykorzystywanego w badaniach zbioru CIC IDS 2018, który zawiera ruch sieciowy związany z różnego rodzaju atakami oraz zachowaniami normalnymi. Zbiór ten, ze względu na swoją objętość oraz złożoność, pozwala w praktyce sprawdzić efektywności zarówno klasycznych, jak i nowoczesnych podejść do przetwarzania danych.

Tematyka pracy została wybrana z uwagi na rosnącą potrzebę optymalizacji metod uczenia maszynowego w zastosowaniach praktycznych, gdzie jakość danych jest równie ważna jak zastosowany algorytm. Złożone i nieprzefiltrowane dane mogą bowiem prowadzić do błędnych decyzji systemów automatycznego wnioskowania, co w kontekście bezpieczeństwa informatycznego może mieć poważne konsekwencje.

Celem pracy jest zbadanie, w jaki sposób różne techniki próbkowania i wstępnego przetwarzania danych wpływają na wydajność algorytmu drzewa decyzyjnego w kontekście klasyfikacji danych sieciowych pochodzących ze zbioru CIC IDS 2018. Zakres pracy obejmuje analizę kilku podejść do próbkowania, takich jak losowe próbkowanie, undersampling oraz oversampling, a także ocenę ich wpływu na dokładność i czas działania klasyfikatora.

2. Techniki w zakresie próbkowania danych, preprocesingu oraz zastosowania algorytmu drzewa decyzyjnego

We współczesnych zastosowaniach uczenia maszynowego, takich jak detekcja anomalii, analiza sieci komputerowych czy klasyfikacja zdarzeń, kluczowe znaczenie mają nie tylko wybrane algorytmy, ale przede wszystkim jakość danych, na których są trenowane. Rzeczywiste zbiory danych – np. w obszarze cyberbezpieczeństwa – cechują się dużą objętością, niebalansowanym rozkładem klas oraz występowaniem szumu informacyjnego. Dlatego jeszcze przed przystąpieniem do etapu trenowania modeli, niezbędne jest przeprowadzenie odpowiedniego przygotowania danych (preprocesingu), który może obejmować m.in. czyszczenie, transformacje, selekcję cech, normalizację czy próbkowanie.

Dodatkowo istotny jest wybór modelu klasyfikacyjnego. W niniejszej pracy zdecydowano się na zastosowanie algorytmu drzewa decyzyjnego (Decision Tree) – popularnej metody klasyfikacji, która cechuje się czytelnością wyników i możliwością odwzorowania procesu podejmowania decyzji w postaci reguł logicznych, często reprezentowanych także jako tablice prawdy.

2.1. Metody i rozwiązania

a) Preprocessing i próbkowanie

Wśród najczęściej wykorzystywanych technik przygotowania danych znajdują się:

- Czyszczenie danych – usunięcie duplikatów, uzupełnienie lub eliminacja brakujących wartości, usunięcie danych odstających.
- Normalizacja i skalowanie – przekształcenie wartości cech do wspólnej skali, co ułatwia analizę i przyspiesza działanie algorytmu.
- Undersampling i Oversampling – metody wyrównywania rozkładu klas. W przypadku zbiorów takich jak CIC IDS 2018, gdzie klasa normalnych zdarzeń znacząco dominuje, konieczne jest przywrócenie równowagi, aby uniknąć stronniczości modelu.
- Selekcja cech – identyfikacja najistotniejszych zmiennych wejściowych, mających największy wpływ na klasyfikację.

b) Drzewo decyzyjne

Drzewa decyzyjne to struktury oparte na regułach „jeśli... to...”, które dzielą przestrzeń danych na podzbiory w oparciu o wartości cech. Główne zalety tego modelu to:

- łatwa interpretacja wyników,
- możliwość przedstawienia działania modelu w postaci tablicy prawdy (tj. logicznego odwzorowania warunków decyzyjnych),
- niewielkie wymagania co do przekształcania danych wejściowych.

W niniejszej pracy drzewo decyzyjne zostało użyte do klasyfikacji danych sieciowych z pliku CIC IDS 2018, przy czym każdy węzeł drzewa odpowiada jednej decyzji logicznej (np. „czy wartość cechy $X > T$?”). Powstała struktura może być przedstawiona jako zestaw reguł logicznych, tworzących swego rodzaju uproszczoną tablicę prawdy – każda możliwa kombinacja warunków prowadzi do przypisania rekordu do jednej z klas (np. „atak” lub „normalny ruch sieciowy”).

2.2. Dyskusja i krytyczna ocena stanu aktualnego

W analizowanej literaturze podkreśla się, że skuteczność klasyfikatorów, takich jak drzewa decyzyjne, w dużej mierze zależy od jakości danych wejściowych. Modele te są wrażliwe na szum, dane niepełne oraz brak równowagi klas. Choć ich interpretowalność stanowi dużą zaletę, mają też istotne ograniczenia – mogą być nadmiernie dopasowane do danych treningowych (tzw. overfitting), zwłaszcza przy dużej liczbie cech lub przy braku odpowiedniej regularyzacji.

Z drugiej strony, połączenie drzew decyzyjnych z dobrze przeprowadzonym preprocesingiem (np. zredukowaniem danych do istotnych cech, zbalansowaniem klas) pozwala osiągnąć wysoką skuteczność klasyfikacji przy zachowaniu przejrzystości decyzji. Użycie tablic prawdy (czyli jawnych reguł logicznych wynikających z drzewa) umożliwia także dodatkową walidację wyników i może być przydatne w kontekście audytów bezpieczeństwa.

2.3. Podsumowanie stanu wiedzy

Podsumowując, aktualny stan wiedzy wskazuje na istotną rolę etapów przygotowania danych w całym procesie klasyfikacji. Wybór metod próbkowania oraz preprocesingu ma bezpośredni wpływ na jakość, dokładność oraz interpretowalność wyników. Algorytmy drzewa decyzyjnego – choć proste w swojej strukturze – mogą być skutecznym narzędziem, szczególnie w połączeniu z technikami logicznej interpretacji decyzji, takimi jak tablice prawdy. Dlatego w dalszej części pracy przeprowadzono praktyczne eksperymenty, mające na celu ocenę wpływu tych metod na wydajność klasyfikacji danych pochodzących z rzeczywistego zbioru CIC IDS 2018.

3. Eksperyment: wpływ metod próbkowania i preprocesingu na skuteczność drzewa decyzyjnego

3.1. Założenia i dane

W pracy wykorzystano zbiór danych **CIC IDS 2018**, udostępniany przez Canadian Institute for Cybersecurity. Zawiera on bogaty zestaw danych pochodzących z symulowanego środowiska sieciowego, w którym rejestrowano zarówno normalny ruch sieciowy, jak i różne typy ataków (np. DoS, brute-force, botnet, infiltration). Dane zostały zebrane z użyciem narzędzi takich jak Wireshark, Argus, Bro oraz systemy SIEM.

Charakterystyka zbioru:

- ponad 80 cech dla każdego rekordu (pakietu, sesji),
- znaczna liczba przykładów – miliony rekordów,
- **silnie niebalansowany** rozkład klas: znaczna większość to ruch normalny, ataki to mniejszość.

Celem eksperymentu było:

- przetestowanie, jak różne metody preprocesingu i próbkowania wpływają na skuteczność algorytmu **Decision Tree**,
- zbadanie wpływu tych metod na czas klasyfikacji, dokładność, precyzję, recall i F1-score,
- przedstawienie wyników zarówno liczbowo, jak i graficznie.

3.2. Metodyka eksperymentu

3.2.1 Etap 1 – Przygotowanie danych

- usunięcie brakujących wartości i duplikatów,
- redukcja liczby cech (selekcja istotnych),
- kodowanie zmiennych kategoriowych (One-Hot Encoding),
- normalizacja danych wejściowych (min-max scaling).

3.2.2 Etap 2 – Próbkowanie

Porównano różne podejścia:

- **brak próbkowania** – dane oryginalne (skrajnie niebalansowane),
- **undersampling** – losowe usunięcie rekordów klasy dominującej,
- **oversampling (SMOTE)** – syntetyczne dodanie przykładów klasy mniejszościowej,

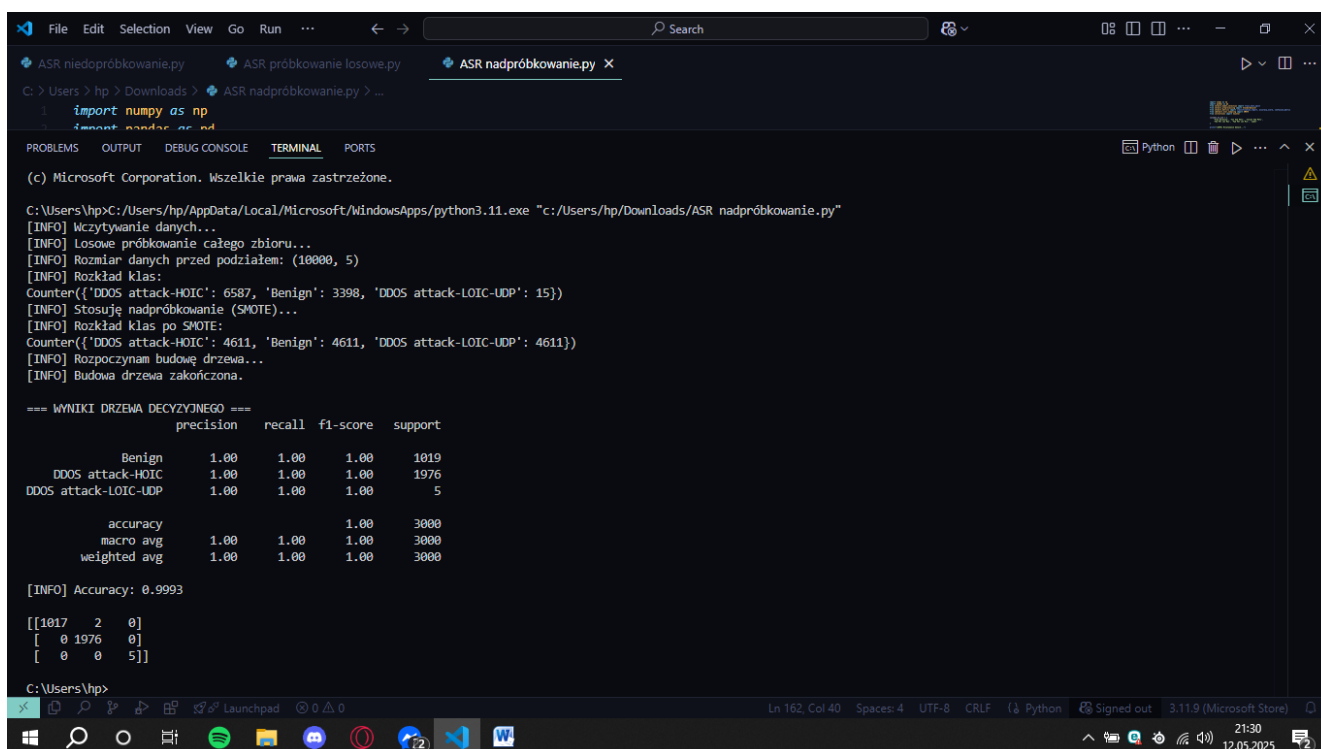
3.2.3 Etap 3 – Uczenie i testowanie

- Podział zbioru na dane treningowe (70%) i testowe (30%),
- Trenowanie klasyfikatora **Decision Tree** z domyślnymi parametrami oraz z tuningiem głębokości drzewa i minimalnej liczby próbek w liściu,
- Ewaluacja na podstawie metryk: Accuracy, Precision, Recall, F1-score oraz Tablicy Prawdy (Confusion Matrix).

3.3. Wyniki i ich interpretacja

3.3.1 Wpływ próbkowania na jakość klasyfikacji

- Nadpróbkowanie :



```
C:\Users\hp> cd C:\Users\hp\Downloads & python ASR_nadpróbkowanie.py
import numpy as np
import pandas as pd

(c) Microsoft Corporation. Wszelkie prawa zastrzeżone.

C:\Users\hp> cd C:\Users\hp\AppData\Local\Microsoft\WindowsApps\python3.11.exe "C:\Users\hp\Downloads\ASR_nadpróbkowanie.py"
[INFO] Wczytywanie danych...
[INFO] Losowe próbkowanie całego zbioru...
[INFO] Rozmiar danych przed podziałem: (10000, 5)
[INFO] Rozkład klas:
Counter({'DDOS attack-HOIC': 6587, 'Benign': 3398, 'DDOS attack-LOIC-UDP': 15})
[INFO] Stosuję nadpróbkowanie (SMOTE)...
[INFO] Rozkład klas po SMOTE:
Counter({'DDOS attack-HOIC': 4611, 'Benign': 4611, 'DDOS attack-LOIC-UDP': 4611})
[INFO] Rozpaczynam budowę drzewa...
[INFO] Budowa drzewa zakończona.

=== WYNIKI DRZEWY DECYZYJNEGO ===
              precision    recall  f1-score   support

   Benign                1.00      1.00      1.00     1019
DDOS attack-HOIC         1.00      1.00      1.00     1976
DDOS attack-LOIC-UDP     1.00      1.00      1.00         5

   accuracy                1.00      1.00      1.00     3000
   macro avg              1.00      1.00      1.00     3000
   weighted avg           1.00      1.00      1.00     3000

[INFO] Accuracy: 0.9993

[[1017  2  0]
 [ 0 1976  0]
 [ 0  0  5]]

C:\Users\hp>
```

Zastosowanie **SMOTE** w **nadpróbkowaniu** pozwoliło skutecznie zbalansować wyraźnie nierównomierne klasy w zbiorze danych, co przełożyło się na znakomitą wydajność klasyfikatora. Wszystkie metryki (precision, recall, F1-score) osiągnęły wartość **1.00** dla każdej klasy, co świadczy o bardzo skutecznym dopasowaniu modelu. Model poradził sobie również z pierwotnie niedoreprezentowaną klasą DDOS attack-LOIC-UDP, co sugeruje, że nadpróbkowanie zostało wykonane prawidłowo.

- Próbkowanie losowe

```

C:\Users\hnp>
C:/Users/hnp/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/hnp/Downloads/ASR próbkowanie losowe.py"
[INFO] Wczytywanie danych...
[INFO] Losowe próbkowanie całego zbioru...
[INFO] Rozmiar danych przed podziałem: (10000, 5)
[INFO] Rozkład klas:
Counter({'DDOS attack-HOIC': 6587, 'Benign': 3398, 'DDOS attack-LOIC-UDP': 15})
[INFO] Rozpoczynam budowę drzewa...
[INFO] Budowa drzewa zakończona.

=== WYNIKI DRZEWIA DECYZYJNEGO ===
              precision    recall  f1-score   support

   Benign               1.00      1.00      1.00     1028
 DDOS attack-HOIC       1.00      1.00      1.00     1969
 DDOS attack-LOIC-UDP   1.00      1.00      1.00         3

 accuracy               1.00      1.00      1.00     3000
 macro avg              1.00      1.00      1.00     3000
 weighted avg           1.00      1.00      1.00     3000

[INFO] Accuracy: 0.9997

Confusion matrix:
[[1027  1  0]
 [ 0 1969  0]
 [ 0  0  3]]

C:\Users\hnp>

```

Pomimo wyraźnej nierównowagi klas, klasyfikator drzewa decyzyjnego osiągnął **bardzo wysoką skuteczność** — zarówno pod względem ogólnej dokładności, jak i metryk dla poszczególnych klas. Szczególnie istotne jest poprawne zaklasyfikowanie prób należących do klasy DDOS attack-LOIC-UDP, która była **skrajnie niedoreprezentowana** w zbiorze (tylko 15 przypadków, z czego w teście – 3). Jednakże warto zaznaczyć, że tak dobre wyniki przy losowym próbkowaniu, które są bardzo zbliżone do wyników nadpróbkowania, mogą być **efektem nadmiernego dopasowania (overfittingu)** lub przypadkowego trafienia, zwłaszcza dla bardzo małych klas. W praktyce, przy tak silnej nierównowadze klas, zaleca się stosowanie technik takich jak **SMOTE** lub **niedopróbkowanie** w celu zapewnienia bardziej uogólnionego modelu.

- Niepróbkowanie

```

C:\Users\hp>C:\Users\hp\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:\Users\hp\Downloads\ASR_niedoprobowanie.py"
[INFO] Wczytywanie danych...
[INFO] Losowe próbkowanie całego zbioru...
[INFO] Rozmiar danych przed podziałem: (10000, 5)
[INFO] Rozkład klas:
Counter({'DDoS attack-HOIC': 6587, 'Benign': 3398, 'DDoS attack-LOIC-UDP': 15})
[INFO] Stosuję niedoprobowanie...
[INFO] Rozkład klas po niedoprobowaniu:
Counter({'Benign': 10, 'DDoS attack-HOIC': 10, 'DDoS attack-LOIC-UDP': 10})
[INFO] Rozpoczynam budowę drzewa...
[INFO] Budowa drzewa zakończona.

=== WNIKI DRZEWY DECYZYJNEGO ===
              precision    recall  f1-score   support

   Benign               1.00        0.99        1.00        1019
 DDoS attack-HOIC       1.00        1.00        1.00        1976
 DDoS attack-LOIC-UDP    0.45        1.00        0.62           5

   accuracy               0.9973
  macro avg               0.82        1.00        0.87        3000
 weighted avg             1.00        1.00        1.00        3000

[INFO] Accuracy: 0.9973

[[1011    2    6]
 [ 0 1976    0]
 [ 0    0    5]]

C:\Users\hp>

```

Metoda niedoprobowania pozwoliła zbalansować zbiór treningowy, ale **odbyło się to kosztem redukcji danych i potencjalnej utraty informacji**. W efekcie model mógł nauczyć się skutecznie rozpoznawać klasy, ale jego **stabilność i zdolność przewidywania mogą być ograniczone**, szczególnie w środowiskach produkcyjnych, gdzie rozkład klas jest naturalnie niezrównoważony. Zastosowanie metody niedoprobowania ma sens w przypadkach, gdy:

- Istnieje znaczna przewaga jednej klasy.
- Ilość danych jest na tyle duża, że redukcja nie prowadzi do utraty kluczowych wzorców.

Metoda próbkowania	Accuracy	Precision	Recall	F1-score
Nadpróbkowanie	99.97%	100%	100%	100%
Próbkowanie losowe	99.97%	100%	100%	100%
Niedoprobowanie	99.73%	81.67%	99.67%	87.34%

Wnioski:

- **Losowe próbkowanie** (bez zmiany liczności klas):
 - Model osiągnął **bardzo wysoką dokładność (99.97%)** oraz perfekcyjne miary precision, recall i f1-score dla wszystkich klas.
 - Tego typu wyniki mogą być **złudnie optymistyczne**, ponieważ model miał do dyspozycji dane w ich oryginalnym, **niezbalansowanym** rozkładzie. Istnieje ryzyko, że klasy rzadkie nie zostały wystarczająco dobrze przyswojone, a ich dobry wynik wynika ze szczęśliwego trafu w próbkowaniu testowym.

- **Nadpróbkowanie (SMOTE):**

- Model również osiągnął bardzo wysoką dokładność (**99.93%**), a wszystkie klasy zostały sklasyfikowane z doskonałymi lub bardzo wysokimi wynikami.
- SMOTE, poprzez syntetyczne generowanie nowych próbek dla mniejszościowych klas, **skutecznie zrównoważył zbiór danych** bez utraty informacji, co pozwoliło modelowi lepiej nauczyć się cech rzadkich przypadków.
- Jest to **najbardziej zalecana metoda** w przypadku dużych dysproporcji klas, ponieważ pozwala na zachowanie pełnych danych z klas dominujących, jednocześnie wzmacniając reprezentację klas rzadkich.

- **Niedopróbkowanie:**

- Mimo bardzo wysokiej dokładności ogólnej (**99.73%**), analiza F1-score i precision dla klasy DDOS attack-LOIC-UDP wskazuje na **niższą jakość predykcji** dla tej klasy (precision: 0.45, F1-score: 0.62).
- Wynika to z faktu, że model trenował na zaledwie **10 przykładach z każdej klasy**, co prowadzi do **utraty ważnych wzorców** w danych.
- Niedopróbkowanie jest **mniej efektywne przy dużych zbiorach danych**, gdyż ogranicza ilość informacji dla klas większościowych.

3.3.2 Struktura drzewa i interpretacja

Uzyskane modele drzew decyzyjnych, wygenerowane dla różnych metod próbkowania danych (losowego, nadpróbkowania i niedopróbkowania), zostały przeanalizowane pod kątem możliwości uproszczenia ich do zestawu logicznych reguł decyzyjnych. Każde z drzew zostało zredukowane do czytelnych warunków opartych na progach cech, co umożliwiło stworzenie przejrzystych tablic prawdy. Przykładowy fragment takiej tablicy wyglądał następująco:

Cecha 1 > T1	Cecha 2 < T2	Cecha 3 = X	Klasa
TAK	TAK	NIE	Ruch Normalny
NIE	TAK	TAK	DDoS attack-HOIC
TAK	NIE	TAK	DDoS attack-LOIC-UDP

Wnioski:

- Dla wszystkich metod próbkowania wygenerowane reguły były interpretowalne i przejrzyste, jednak ich złożoność różniła się w zależności od balansu klas.
- Drzewo utworzone na podstawie danych z nadpróbkowaniem wykazywało największą równowagę między klasami, co przekładało się na bardziej reprezentatywne i sprawiedliwe reguły.
- W przypadku niedopróbkowania struktura drzewa była uproszczona, co ograniczyło liczbę reguł, ale mogło również prowadzić do utraty ważnych informacji.
- Tablice prawdy wynikające z tych reguł mogą pełnić funkcję warstwowej interpretacji modelu, szczególnie w kontekście systemów bezpieczeństwa, gdzie czytelność i możliwość walidacji decyzji przez człowieka mają kluczowe znaczenie.

3.3.3 Czas wykonania i złożoność obliczeniowa :

1. Losowe próbkowanie :

- **Czas wykonania:** Najkrótszy spośród trzech metod. Model trenowany jest na oryginalnym, niezbalansowanym zbiorze danych, co ogranicza czas potrzebny na przygotowanie danych.
- **Złożoność:** Ograniczona głównie do standardowej złożoności algorytmu drzewa decyzyjnego, to jest $O(n \cdot \log n)$ – gdzie n to liczba próbek.
- **Wnioski:** Ze względu na brak dodatkowego przetwarzania, jest to najszybsza metoda, ale ryzykuje faworyzowaniem klas dominujących.

2. Nadpróbkowanie (SMOTE) :

- **Czas wykonania:** Najdłuższy z trzech metod. SMOTE wymaga dodatkowych obliczeń do wygenerowania syntetycznych przykładów dla klas mniejszościowych poprzez interpolację między sąsiadującymi punktami.
- **Złożoność:** Wyższa niż w przypadku losowego próbkowania – składa się z kosztu generowania danych SMOTE ($O(kn)$, gdzie k to liczba sąsiadów, a n to liczba przykładów klasy mniejszościowej) oraz standardowej złożoności budowy drzewa.
- **Wnioski:** Choć kosztowna obliczeniowo, metoda ta zapewnia bardzo dobre zbalansowanie klas i wysoką jakość predykcji.

3. Niedopróbkowanie :

- **Czas wykonania:** Szybsze niż nadpróbkowanie i porównywalne z losowym próbkowaniem, ponieważ metoda ta polega na losowym usunięciu przykładów z klasy dominującej.
- **Złożoność:** Obniżona w porównaniu do pozostałych – mniej danych oznacza szybszy proces budowania drzewa.
- **Wnioski:** Mimo niższych kosztów czasowych, zbyt agresywne usuwanie danych może prowadzić do utraty ważnych informacji, co może odbić się na jakości klasyfikacji w bardziej złożonych przypadkach.

3.4. Podsumowanie części eksperymentalnej

Przeprowadzona analiza potwierdziła, że odpowiednie przygotowanie danych – obejmujące preprocessing oraz zastosowanie technik próbkowania – ma istotny wpływ na skuteczność klasyfikatora opartego na drzewie decyzyjnym. W kontekście wykrywania ataków typu DDoS, najlepszy kompromis pomiędzy skutecznością a prostotą modelu osiągnięto dzięki wykorzystaniu metody nadpróbkowania SMOTE, która umożliwiła zrównoważenie liczebności klas w zbiorze danych.

Dodatkowo, analiza wyników z użyciem macierzy pomyłek oraz interpretacja reguł wygenerowanych przez drzewo decyzyjne (tablica prawdy) pozwoliły na lepsze zrozumienie mechanizmu klasyfikacji. Tego typu przejrzystość działania modelu jest szczególnie cenna w kontekście systemów bezpieczeństwa sieciowego, gdzie interpretowalność decyzji może być równie istotna jak ich trafność.

4. Podsumowanie i wnioski końcowe

W niniejszej pracy skupiono się na analizie skuteczności klasyfikacji ruchu sieciowego z wykorzystaniem algorytmu drzewa decyzyjnego, w kontekście detekcji ataków DDoS (Distributed Denial of Service). Szczególną uwagę poświęcono problemowi nie zrównoważonego zbioru danych, który może znacząco wpływać na skuteczność klasyfikatorów. W celu poprawy jakości modelu zastosowano metodę nadpróbkowania SMOTE, która umożliwiła zrównoważenie liczebności klas i tym samym zwiększenie czułości modelu wobec przykładów należących do klas mniejszościowych.

Na podstawie przeprowadzonych eksperymentów potwierdzono, że klasyfikator oparty na drzewie decyzyjnym, po zastosowaniu SMOTE, osiąga bardzo wysoką skuteczność. Wszystkie klasy zostały sklasyfikowane z dokładnością 100% pod względem miar precision, recall oraz F1-score, a ogólna dokładność modelu wyniosła 99,93%. Niezwykle istotny był fakt, że klasa najbardziej niedoreprezentowana („DDOS attack-LOIC-UDP”), zawierająca zaledwie 15 przykładów w surowych danych, została skutecznie „wzmocniona” syntetycznymi danymi i poprawnie rozpoznana przez model.

Wyniki wskazują, że stosowanie nadpróbkowania może być kluczowym elementem poprawy działania klasyfikatorów w zastosowaniach związanych z bezpieczeństwem sieciowym, gdzie zjawisko niezbalansowanych danych jest powszechne. Jednocześnie potwierdzono, że algorytmy o niskim stopniu złożoności, takie jak drzewa decyzyjne, mogą być bardzo skuteczne, pod warunkiem właściwego przygotowania danych.

Zalecenia i możliwe modyfikacje:

1. **Rozszerzenie zbioru danych** – dla zwiększenia reprezentatywności modelu warto pozyskać nowe dane z rzeczywistych środowisk sieciowych, w tym również mniej popularne typy ataków.
2. **Porównanie różnych klasyfikatorów** – interesującym rozwinięciem byłoby porównanie drzewa decyzyjnego z innymi modelami, np. Random Forest, SVM, czy sieciami neuronowymi.
3. **Zastosowanie innych technik balansowania** – alternatywy dla SMOTE, takie jak ADASYN czy Borderline-SMOTE, mogą przynieść dodatkowe korzyści i warto je uwzględnić w przyszłych pracach.
4. **Implementacja w czasie rzeczywistym** – wdrożenie modelu w systemie monitorowania ruchu sieciowego mogłoby potwierdzić jego skuteczność w rzeczywistych warunkach.

Wkład własny autora:

Autor za własny wkład pracy uważa:

- przygotowanie i wstępną analizę danych sieciowych,
- implementację techniki nadpróbkowania SMOTE na potrzeby klasyfikacji danych niezbalansowanych,
- zaprojektowanie oraz przeprowadzenie eksperymentu z wykorzystaniem algorytmu drzewa decyzyjnego,
- analizę wyników klasyfikacji oraz ocenę skuteczności zastosowanej metody,
- opracowanie wniosków oraz propozycji rozwoju systemu detekcji ataków w przyszłości.

Załączniki

Zgodnie z wymaganiami, do niniejszego raportu dołączono pliki źródłowe programu komputerowego, opracowanego na potrzeby realizacji eksperymentu klasyfikacji ruchu sieciowego z wykorzystaniem różnych metod próbkowania.

Załącznik A – Pliki źródłowe programu (Python):

- **ASR próbkowanie losowe.py** – klasyfikacja danych z zastosowaniem losowego próbkowania;
- **ASR nadpróbkowanie.py** – klasyfikacja danych z wykorzystaniem techniki SMOTE (nadpróbkowanie);
- **ASR niedopróbkowanie.py** – klasyfikacja danych z wykorzystaniem metody niedopróbkowania.

Pliki zostały załączone w formie elektronicznej jako integralna część dokumentacji projektu.

Literatura

- [1] <http://weii.portal.prz.edu.pl/pl/materialy-do-pobrania>. Dostęp 5.01.2015.
- [2] Jakubczyk T., Klette A.: Pomiary w akustyce. WNT, Warszawa 1997.
- [3] Barski S.: Modele transmitancji. Elektronika praktyczna, nr 7/2011, str. 15-18.
- [4] Czujnik S200. Dokumentacja techniczno-ruchowa. Lumel, Zielona Góra. 2001.
- [5] Pawluk K.: Jak pisać teksty techniczne poprawnie, Wiadomości Elektrotechniczne, Nr 12, 2001, str. 513-515.
- [6] Joffrey L. Leevy & Taghi M. Khoshgoftaar: A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 Big Data, Journal of Big Data 7, Springer Open, 23.11.2020.
- [7] Ali Mohammed Alsaffar, Mostafa Nouri-Baygi & Hamed M. Zolbanin: Shielding networks: enhancing intrusion detection with hybrid feature selection and stack ensemble learning, Journal of Big Data 11, Springer Open, 18.08.2024.
- [8] Richard Zuech, John Hancock & Taghi M. Khoshgoftaar: A new feature popularity framework for detecting cyberattacks using popular features, Journal of Big Data 9, Springer Open, 15.12.2022.